Knapsack Problem

Unbounded knapsack
0-1 knapsack

Readings and similar problems

- Reading
  - http://algorithmics.lsi.upc.edu/docs/Dasgupta-Papadimitriou-Vazirani.pdf
    - Chapter 6
- Similar problems
  - https://www.geeksforgeeks.org/subset-sum-problem-dp-25/
  - https://leetcode.com/problems/partition-equal-subset-sum/description/
  - https://leetcode.com/problems/partition-equal-subset-sum/description/
  - https://leetcode.com/problems/ones-and-zeroes/description/

0-1 Knapsack Problem

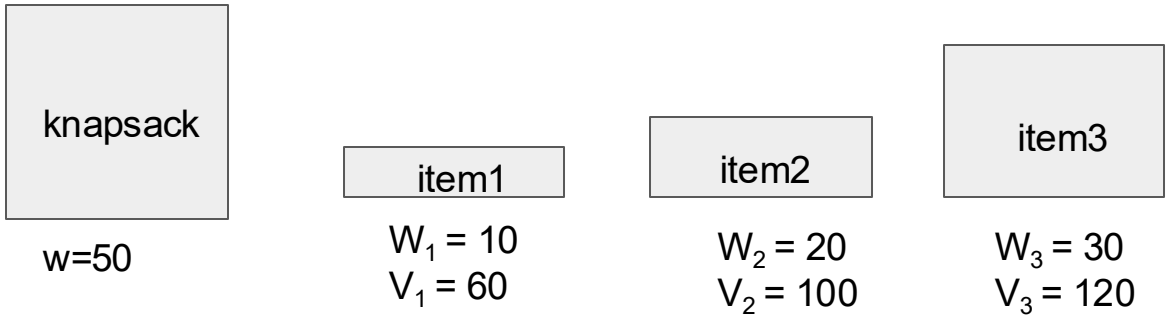
- **Input**:
  - n items with weight w_1, w_2, …, w_n and values v_1,v_2, … ,v_n
  - A knapsack with capacity W (total weight of items at most W)

- **Output**:
  - Subset **S** of items fit in the knapsack: $\sum_{i \in S} w_i \leq W$ and maximizes the total value $\sum_{i \in S} v_i$

0-1 Knapsack Problem

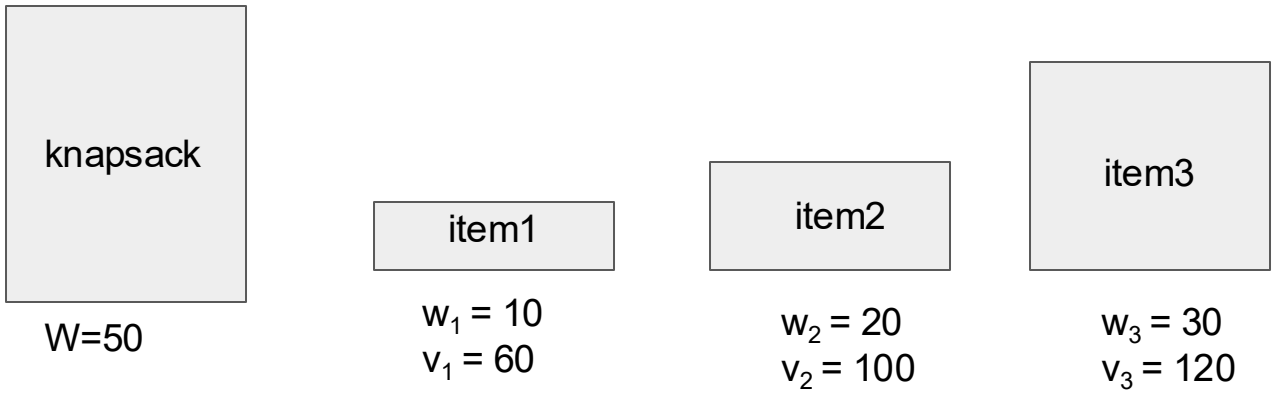
- **Brute-force**
  - Generate all the subsets of the set of n items
  - Compute the total weight of each subset
    - To identify subsets with weight <= W
  - Find a subset that has maximum value
- **Runtime: Ω (2^n)**
  - Number of subsets of a set with n elements are 2^n
    - Why?
  - Inefficient

Subset	Total weight	Total value
{}	0	0
{1}	10	60
{2}	20	100
{3}	30	120
{1,2}	30	160
{2,3}	50	220
{1,3}	30	160
{1,2,3}	60	280



Unbounded Knapsack

- **Input**:
  - n items with weight w_1, w_2, …, w_n and values v_1,v_2, … ,v_n
  - A knapsack with capacity W (total weight of items at most W)
  - The are unlimited numbers of each item i
- **Output**: Subset **S** of items fit in the knapsack $\sum_{i \in S} w_i \leq W$ and maximizes the total value $\sum_{i \in S} v_i$



Unbounded Knapsack

- **Brute force**
  - Every item i can be used at most $\lfloor W/w_i \rfloor$ times
    - As long as $kw_i <= W$
    - W is knapsack capacity
    - w_i is the weight of item i
  - For every item there are 0… k possibilities, k = $\lfloor W/w_i \rfloor$
  - Number of configurations: k * k * k … k = k^n
  - For each configuration, calculate the total weight and value: O(n)
  - Runtime : **Ω** (k^n) or O ($n\ k^n$)
    - Exponential

Unbounded Knapsack

- Possible subproblems:
  - **Knapsack of smaller capacities**
  - Fewer items to choose from

Unbounded Knapsack: recursive formulation

- Subproblem:
  - Knapsack of smaller capacities
  - S(W): optimal solution (optimal value) for capacity W
- Recursive formula

S(W) = max { optimal solution to the knapsack with capacity w_i + v_i }

$1 \le i \le n$ and $w_i <= W$

$$S(W) = \max(S(W-w_i)+v_i)$$

$1 \le i \le n$ and $w_i <= W$

Unbounded Knapsack: Optimal substructure

- Subproblems:
  - Knapsack of smaller capacities
- Optimal Substructure
  - **S(W):** Optimal Solution for a knapsack of size W
  - If **S(W)** contains the i-th item, then **S(W) - { item i }** is the optimal solution to the subproblem **S(W-w_i)**
    - If not, we could have replaced **S(W-w_i)** with the optimal solution and get a solution better than **S(W)**: **contradiction**, S(W) is the optimal solution
  - The problem has the optimal substructure property

Unbounded Knapsack: Recursive Implementation

```
def unboundedK(W, n, w[0 … n], v[0… n]):

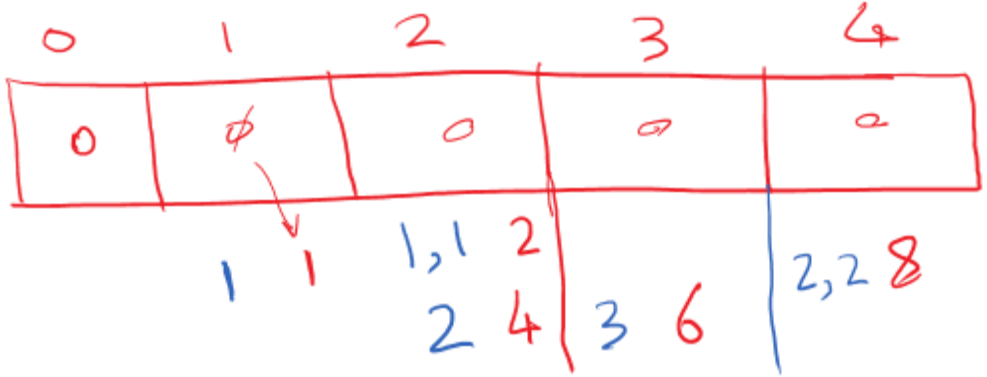
    if W==0:
        return 0

    q = -∞
    for i in range(n):
        if w[i] <= W:
            q = max(q, unboundedK(W-w[i], n, w, v) + v[i] )
    return q
```

Runtime: Left as an exercise

Memoization: Left as an exercise

Unbounded Knapsack: Bottom-up example



$S[4] = S[4-1] + V_1 = 6+1 = 7$ 3, 1

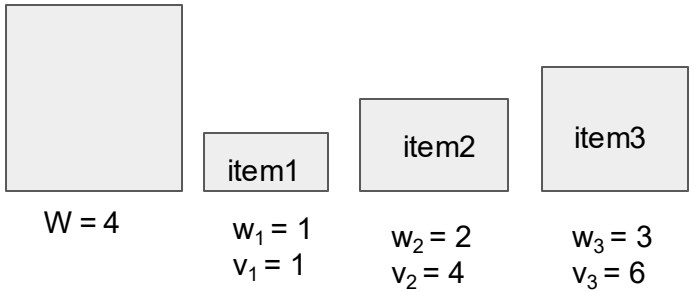
$S[4] = S[4-2] + V_2 = 4+4 = 8$ 2, 2

$S[4] = S[4-3] + V_3 = 1+6 = 7$ 1, 3

$S[3] = S[3-1] + V_1 = 5$ 2, 1

$S[3] = S[3-2] + V_2 = 5$ 1, 2

$S[3] = S[3-3] + V_3 = 6$ 3



Unbounded Knapsack: DP: bottom-up

```
def UnboundedKnapsack(W, n, w[0 … n], v[0… n])
    S[0] = 0
    for j = 1 to W:
        S[j]=0
        for i = 1 to n:
            if w[i] ≤ j:
                S[j] = max(S[j-w[i]] + v[i], S[j])
     return S[W]
```

Runtime: A nested loop → **O(nW)**

Exercise: Modify the code to contain the solution

Unbounded Knapsack: DP: bottom-up

```
def UnboundedKnapsack(W, n, w[0 … n], v[0… n])
    S[0] = 0
    for j = 1 to W:
        S[j]=0
        for i = 1 to n:
            if w[i] ≤ j:
                S[j] = max(S[j-w[i]] + v[i], S[j])
     return S[W]
```

P[0] = [ ]

P[j] = P[j - w[i]] . append(i)

Runtime: A nested loop → **O(nW)**

Exercise: Modify the code to contain the solution

0-1 Knapsack Problem

- Subproblems:
  - Fewer number of items
  - Knapsack with fewer size
- Recursive relation for unbounded knapsack does not work
  - here each item can be used at most once
  - So, when solving the subproblem we need to know what items are available
  - Therefore, we add another dimension to our recursive relation (and the table to store the results)

0-1 Knapsack Problem: Optimal Substructure Property

- Optimal Substructure
  - **S(W, n):** Optimal Solution for a knapsack of size **W** and **n** items
  - If **S(W, n)** contains item n, then **S(W, n) - { item n }** is the optimal solution to the subproblem **S(W-w_n, n-1)**
    - If not, we could have replaced **S(W-w_n, n-1)** with the optimal solution and get a solution better than **S(W, n)**: **contradiction**, **S(W, n)** is the optimal solution
  - If **S(W, n)** does not contains the item n, then **S(W, n)** is the optimal solution to the subproblem **S(W, n-1)**
    - If not, we could have replaced **S(W, n-1)** with the optimal solution and get a solution better than **S(W, n)**: **contradiction**, **S(W, n)** is the optimal solution
  - → The problem has the optimal substructure property

0-1 Knapsack Problem: Recursive Formula

- S(W, n): Optimal solution for knapsack with capacity W and n items.
-

$$S(W, n) = \begin{cases} 0 & \text{if } n=0 \text{ or } W=0 \\ S(W, n-1) & \text{if } w_n > W \\ \max \{ S(W, n-1) , S(W-w_n , n-1) + v_n \} & \end{cases}$$

0-1 Knapsack Problem: Recursive Formula

- S(W, n): Optimal solution for knapsack with capacity W and n items.
-

$$S(W, n) = \begin{cases} 0 & \text{if } n=0 \text{ or } W=0 \\ S(W, n-1) & \text{if } w_n > W \\ \max \{ S(W, n-1) , S(W-w_n , n-1) + v_n \} & \end{cases}$$

optimal solution does not use item n

optimal solution uses item n

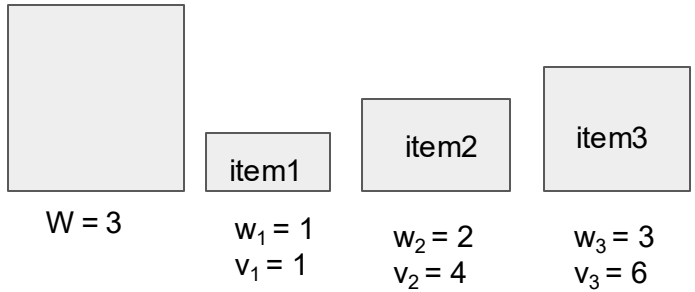
n

0-1 Knapsack: Bottom-up example

W \ n	0	1	2	3
0	0	0	0	0
1	0	1	1	1
2	0	1	4	4
3	0	1	5	6

$S[3,3] = S[0,2] + 6$

$S[3,2]$



0-1 Knapsack Problem: DP: bottom-up

```
def knapsack(W, n, w[1…n], v[1…n])
    for j=0 to W:
        S[j, 0] = 0
    for i=0 to n:
        S[0, i] = 0

    for j=1 to W
        for i=1 to n
            S[j, i]= S[j, i-1]
            if w[i] <= j:
                t = S[j-w[i], i-1] + v[i]
                if t > S[j, i]
                    S[j, i] = t

     return S[W, n]
```

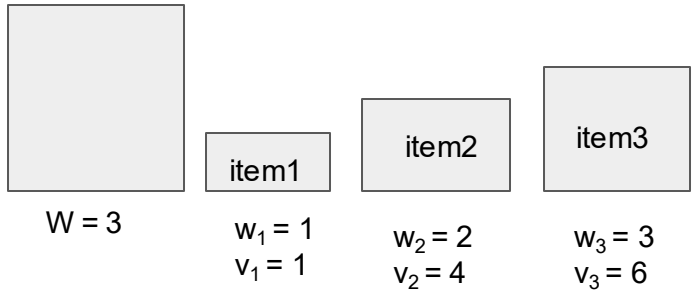
0-1 Knapsack: Bottom-up example

w \ n	0	1	2	3
0	0	0	0	0
1	0	1	1	1
2	0	1	4	4
3	0	1	5	6
		1	1,2	3

$$S[3,2] = \max \begin{cases} S[3,1] = 1 \\ S[3-2,1] + 4 = 5 \end{cases}$$

$$S[3,3] = \max \begin{cases} S[3,2] = 5 \\ S[3-3,1] + 6 = 6 \end{cases}$$

$$S[3,1] = \max \begin{cases} S[3,0] = 0 \\ S[3-1,0] + v_1 = 1 \end{cases}$$



0-1 Knapsack Problem: Finding the solution

```
def knapsack(W, n, w[1…n], v[1…n])
    for j=0 to W:
        S[j, 0] = 0
        P[j, 0] = []
    for i=0 to n:
        S[0, i] = 0
        P[0, i] = []
    for j=1 to W
        for i=1 to n
            S[j, i]= S[j, i-1]
             P[j, i] = P[j, i-1]
            if w[i] <= j:
                t = S[j-w[i], i-1] + v[i]
                if t > S[j, i]
                    S[j, i] = t
                     P[j, i] = P[j-w[i], i-1] ∪ i
     return S[W, n]
```